

1° a) Premikanje se po parih a -ja in b -ja vz pomoč 4 spremenljivke ki označujejo do katerega indeksa smo prišli (začetni ind para iz a , začetni iz b , končni iz a in končni iz b). Primenjamo z funkcijo `compare` začetne indekse za določeno tistega ki je krajši (a ali b par). Če funkcija vrne karkoli različno od 0, to vrnemo, v nasprotnem primeru premikamo začetka naprej. Ko je začetni indeks A enak končnemu indeksu A , gremo na naslednji par Evako tudi za b . Če po koncu primenjajna funkcija ne vrne nič, preverimo ali so vsi pari v obeh vektorjih primenjani. Če so, vrnemo 0. Če je en a večji vrnemo 1, drugače -1.

Prostorska zahtevnost je odvisna od števila parov, $O(n)$. n -št. parov v večjem vektorju.

Časovna $O(1)$ ker zraven vhodnih podatkov ne hranimo dodatne. vseeno katero upišiš

Časovna pa je $O(t \cdot L)$ kjer je L število znakov v krajšem sestavljenem stringu od parov

b) `int compareMulti(string s, vector<PII> &a, vector<PII> &b) {`
`int i, j = 0;`

zakaj ni n^2 ? Ker se zvebiš po enega para ko ga primenjáš/obdeláš!

`auto [c, d] = a[0]; int zacA = c; int konA = d;`

`auto [e, f] = b[0]; int zacB = e; int konB = f;`

`while (i < a.size() && j < b.size()) {`

`int l = min(konA - zacA, konB - zacB)`

`int vrni = compare(zacA, l, zacB, l);`

`if (vrni != 0) return vrni;`

`zacA++; zacB++;`

`if (zacA == konA) {`

`i++; if (i == a.size()) break;`

`auto [x, y] = a[i];`

`zacA = x;`
`konA = y;`

`if (zacB == konB) {`

`j++; if (j == b.size()) break;`

`auto [x, y] = b[j];`

`zacA = x;`
`konA = y;`

`if (i == a.size() && j == b.size()) return 0;`

`if (i == a.size() && j < b.size()) return 1;`

`else return -1;`

2° Kruskalov algoritem =>

a) Število povezav za linearni seznam oziroma pot je $e = u - 1$

operacije v disj. množ., zanemarljive
 $O(e \log e + e) = O(e \log e)$

Časovna zahtevnost Kruskalovog algoritma je odvisna od števila povezav v grafu. Najbolj časovno zahtevan del je urejanje povezav, ker urejanje vzame $O(e \log e)$ časa, je to tudi celotna časovna zahtevnost algoritma. $O(e \log e) = O((u-1) \log(u-1)) \approx O(u \log u)$. Linearni seznam je sam po sebi že minimalno vpeto drevo.

$$u \log u - 1 \log u \rightarrow O(u \log u)$$

b) V ciklu je $e = u$ povezav. Čas zahtevnost odvisna od urejanja povezav => $O(u \log u)$.

V min vpetemu drevesu ni ciklov, zato bo ena povezava manj kot v ciklu ($u-1$).

Algoritem mora eno povezavo odstraniti, ampak se vedno jih mora vnesti u .

c) Drevo z u vozlišči ima ugrez $u-1$ robov (ne glede na stopnjo vozlišča). Časovna zahtevnost je torej $O(e \log e) = O((u-1) \log(u-1)) \approx O(u \log u)$

d) V gostih grafih je število vozlišč velikostnega reda, ki je blizu maksimalnemu št. povezav.

$e = O(u^2)$, oziroma v našem primeru $e = \frac{u \cdot (u-1)}{2} \approx u^2$ (ker je graf nesuspen)

$$O(e \log e) = O(u^2 \log u^2) = O(u^2 \cdot 2 \log u) = O(u^2 \log u) \quad \text{pomembno}$$

$$e = 2vs - v - s \approx v \cdot s \quad O(e \log e) = O(vs \cdot \log(vs))$$

3° Skip list

a) Problem, ki ga rešuje preskočni seznam, je počasi iskanje, vstavljanje in brisanje elementov v navadnem povezanem seznamu, kjer so operacije v najslabšem primeru $O(n)$. Preskočni seznam izboljša učinkovitost teh operacij s tem, da omogoča hitrejše iskanje v povprečnem času $O(\log n)$.

Kotiko med preskočnim seznamom in dodajanje kazalcev (na vsaka 2, 4, ...) in navadnem povezanem seznamu je dinamičnost strukture.

b) Preskočni seznam je sestavljen iz več nivojev urejenih povezanih seznamov, kjer vsak višji nivo vsebuje podmnožico elementov iz nižjih nivojev. Vsako vozlišče vsebuje kazalce na naslednje vozlišče v svojem nivoju in na vozlišče v nivoju nižje. Vsi nivoji so urejeni naraščajoče.

Iskanje elementov poteka tako, da začnemo na najvišjem nivoju in se premikamo po kazalcih naprej, dokler ne najdemo vozlišča z vrednostjo $>$ od iskane ali pridemo do konca nivoja. Nato se spustimo en nivo nižje in postopek ponovimo. Tako nadaljujemo, dokler element ne najdemo ali ugotovimo, da ga ni v seznamu. Zaradi tega preskakovanja segmentov se čas. zalet.

iskanja v povprečju zmanjša na $O(\log n)$. Zaradi vključnosti dodajanja nivojev pa se nam lahko zgodi  takšna situacija, ki zato je najslabša časovna zahtevnost iskanja $O(n)$.

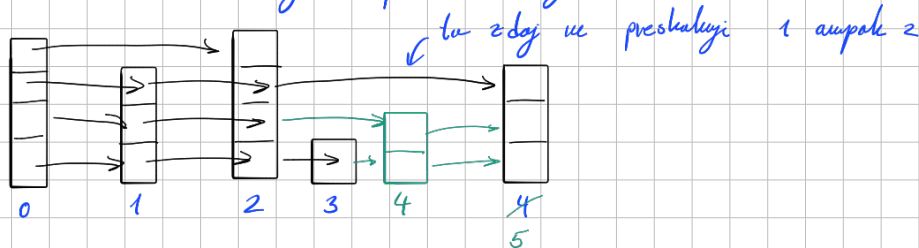
Pri dodajanju elementa najprej na enak način poiščemo ustrezno mesto, nato pa element vstavimo na osnovni nivo. Nato z verjetnostjo p "dvignemo" ta element v višje nivoje, kjer ga vstavimo na enak način. Zaradi istega argumenta kot zgoraj čas. zalet dodajanja elementa v najslabšem primeru $O(n)$.

c) Za odstranitev elementa v preskočnem seznamu najprej izvedemo iskanje, kjer si na vsakem nivoju shranimo predhodno vozlišče iskanega vozlišča. Ko najdemo iskano vozlišče, uporabimo te prehodnike, da spremenimo kazalce tako, da preskočijo vozlišče, ki ga želimo odstraniti. Ta postopek ponovimo na vseh nivojih, kjer iskano vozlišče obstaja.

Če po odstranitvi na višjih nivojih ni več nobenega vozlišča, se ti nivoji lahko logično izpraznijo, vendar struktura preskočenega seznama obstaja. Ker iskanje v uvisl. primeru traja $O(u)$ je tudi operacija odstranjevanja $O(u)$.

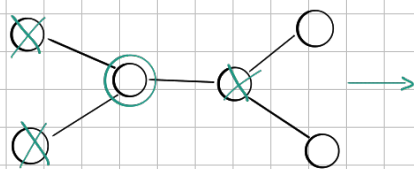
d) Vsaki korekci v preskočnem seznamu razširimo tako, da poleg povezave na naslednjo vozlišče hranimo tudi število preskočenih elementov. Vse operacije v povprečju, $O(\log u)$. Spet gremo po uvisljem nivoju, preštejemo šb. elementov ki so preskočeni, če bi naslednji preskočil preveč, gremo nižje.

ko uvidimo ustrezno mesto dodamo element in spreminjamo šb. preskočenih elementov. Pri brisanju pa zmanjšamo.



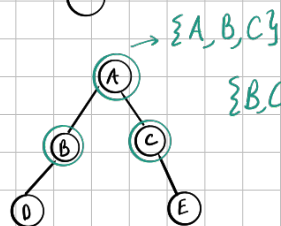
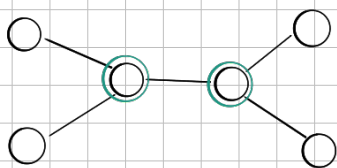
4°

a) protiprimer



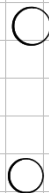
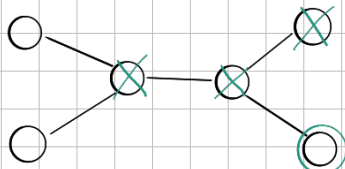
aciklični povezan graf $\rightarrow$ drevo

b)



$\{B, C\}$ pa je minimalno

c)



X

vzeli smo starša v min dominacijsko mn.

$\min(f(k, 0), f(k, 1))$ ali

nisimo vzeli starša

d) dinamično programiranje gredy metoda ne deluje!

